Reduced-Order Models for a Simscape Boost Converter

Neural ODE, LSTM-NARX, and State-Space Estimation Approaches

March 2026

Best result: 47x speedup with Vout RMSE < 0.05V

Algorithms and tricks applicable to any Simscape/power-converter model

Table of Contents

1. Introduction 3
2. Branch A: LSTM-NARX 4
3. Branch B: State Space Estimation 6
4. Branch C: PyTorch Neural ODE 9
   4.1 Architecture & Normalization 9
   4.2 Curriculum Training 11
   4.3 Custom Loss Functions 12
   4.4 Ablation Study 13
   4.5 Extended Training & Results 14
5. Branch C: MATLAB Neural State Space 15
   5.1 Issues vs PyTorch 15
   5.2 Fix Attempts 17
6. PyTorch to Simulink Deployment 18
7. Comparison & Timing Results 20
8. Repository File Listing 22

1. Introduction

This report documents the development of reduced-order models (ROMs) for a Simscape switching boost converter. The goal is to replace the computationally expensive Simscape model with a fast surrogate that captures both averaged dynamics (voltage/current envelopes) and switching-frequency ripple, enabling real-time simulation and hardware-in-the-loop testing.

Generalizability: The ROM workflow described here (data collection, training, validation, deployment) applies to any Simscape model, not just boost converters. The tricks and algorithms are noted throughout as generalizable or topology-specific.

Boost Converter Specifications

Parameter	Value
Input voltage (Vin)	5 V
Inductance (L)	20 uH
Output capacitance (C)	1480 uF
Switching frequency	200 kHz
LC resonant frequency	~600 Hz
Duty cycle range (training)	0.125 - 0.75
ROM sample time	5 us

ROM Challenge

The boost converter operates at 200 kHz PWM, creating inductor current ripple at the switching frequency. The LC resonance at ~600 Hz causes underdamped oscillations after duty cycle step changes. An effective ROM must:

1. Track the voltage/current envelopes across the full duty cycle range
2. Capture the LC oscillatory transient after step changes
3. Optionally reproduce the switching ripple
4. Run >10x faster than Simscape

Three approaches were explored: LSTM-NARX (Branch A), linear state-space estimation (Branch B), and Neural ODE / Neural State-Space (Branch C).

2. Branch A: LSTM-NARX

Architecture

Branch A uses a Nonlinear AutoRegressive with eXogenous inputs (NARX) architecture implemented with a PyTorch LSTM:

```
Input: [duty(t), Vout(t-1)]  (2 features)
LSTM: 1 layer, 32 hidden units
Output: Vout(t)  (1 output)
Total parameters: ~4,500
```

The NARX feedback means Vout(t-1) is fed back as input, creating an autoregressive loop. During training, the true Vout(t-1) is used (teacher forcing). During inference, the model uses its own predicted Vout(t-1), which can cause error accumulation.

Key Techniques

TRICK (Generalizable): DC Gain Pre-Training: Before sequence training, pre-train the LSTM on steady-state operating points from Branch B identification. This anchors the model to the correct DC relationship before learning dynamics. For any converter, first establish the input-output DC operating curve, then train on transient sequences.

TRICK (Generalizable): Sequence Length = 20ms (4000 samples at 5us): Chosen to cover ~12 full LC oscillation cycles at 600 Hz, ensuring the LSTM sees the complete ring-down after a duty step change. For any system, set the sequence length to 10-15x the dominant oscillation period.

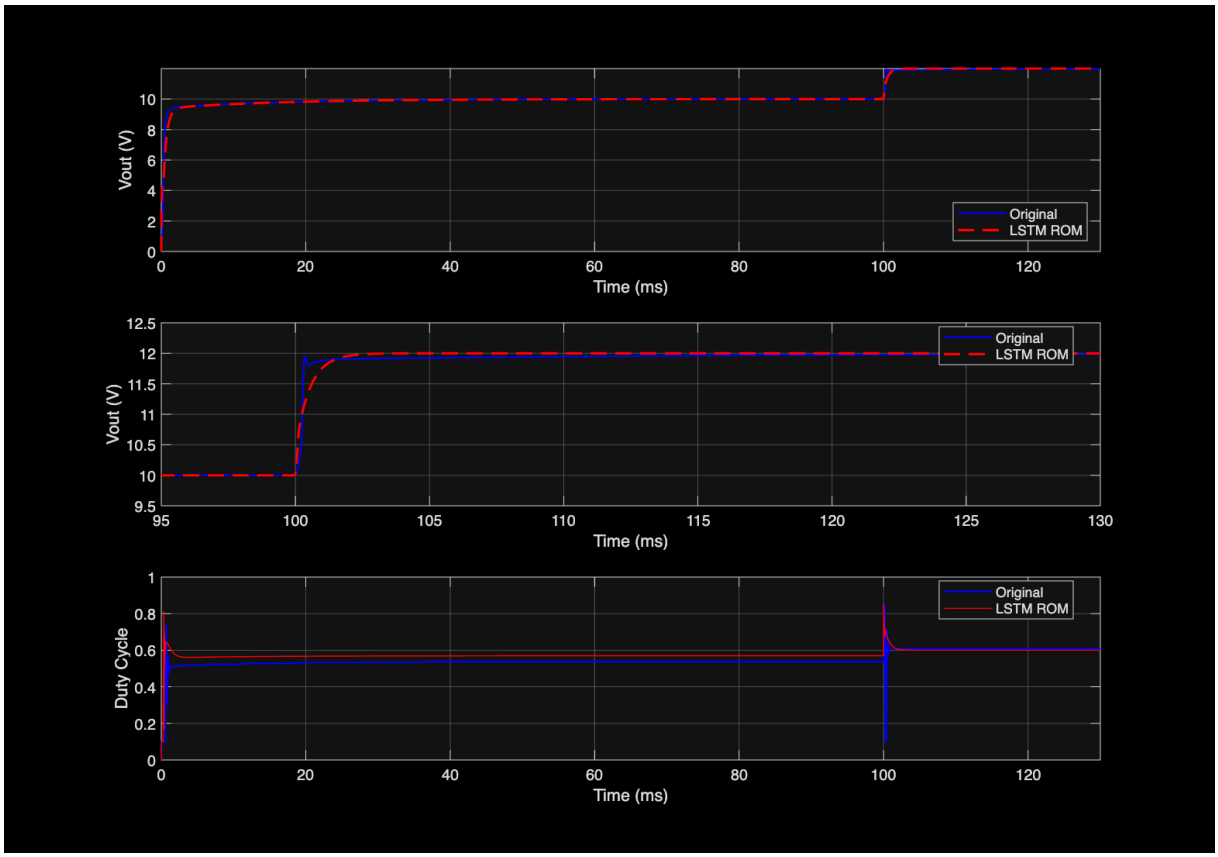
Training used Adam optimizer with cosine annealing LR schedule, gradient clipping to 1.0, and early stopping with patience=30 epochs. The V2 model added longer sequences and the DC gain pre-training insight from Branch B.

Results and Limitations

The LSTM-NARX achieved reasonable Vout tracking but suffered from:

1. Autoregressive drift: small errors compound over long simulations
2. Single-output: only Vout was predicted, not inductor current (iL)
3. No physical constraints: the model has no knowledge of circuit dynamics

These limitations motivated Branch C with the Neural ODE approach, which models dx/dt directly and predicts both Vout and iL simultaneously.


Vout (V)
Time (ms)
Original
LSTM ROM
Vout (V)
Time (ms)
Original
LSTM ROM
Duty Cycle
Time (ms)
Original
LSTM ROM

3. Branch B: State Space Estimation

Two identification approaches were explored to extract linear models at multiple duty cycle operating points. Both produce A, B, C, D matrices at each grid point which are then interpolated to form an LPV (Linear Parameter-Varying) model.

Approach 1: PRBS + ssest (Time Domain)

Script: branch_b_lpv/collect_openloop_id_data.m

At each operating point (D = 0.15, 0.25, ..., 0.85):

1. Apply a small PRBS perturbation (+-5% duty) around the DC operating point
2. Record the Vout and iL time-domain response
3. Pass iddata(dVout, du, Ts) to ssest to identify a 2nd-order state-space model

Simple and direct, but relies on ssest to separate dynamics from switching noise in the raw time-domain data. No post-processing corrections applied.

Approach 2: frestimate + tfest (Frequency Domain) -- USED IN FINAL ROM

Script: branch_b_lpv/frest_openloop_id.m

At each operating point (D = 0.15, 0.20, ..., 0.70), run in parallel:

1. DC gain measurement: two constant-duty steady-state simulations
2. frestimate: inject known frequencies, measure FRD (frequency response data)
3. tfest: fit 2nd-order TF to FRD with np=2, nz=0 (forced by LC count)
4. PEM refinement: refine on step response to correct transient shape
5. DC gain correction: force TF DC gain to match steady-state measurement
6. iL sign/magnitude verification: re-check iL DC gain serially after parfor
7. Convert TF to state-space: ss(tfest_model) gives A, B, C, D matrices

The A matrices at each operating point are used as Jacobian targets in PyTorch Neural ODE training (Jacobian matching loss: MSE(df/dx, A_ssest)).

Generalizability: Both approaches work with any Simulink/Simscape model. The technique of sweeping a scheduling parameter and fitting models at each operating point is the standard approach for building LPV models of nonlinear systems.

Why Approach 2 Outperforms Approach 1

1. DC gain correction (biggest factor): frestimate at low frequencies is corrupted by switching harmonics, giving wrong DC gain. Approach 2 measures DC gain from separate steady-state simulations and forces the TF to match. Approach 1 has no such correction -- ssest fits the raw PRBS data including low-frequency errors.

2. PEM refinement: after tfest, a second pass with pem on step response data corrects both DC gain and transient shape simultaneously. Approach 1 is single-pass.

3. Frequency domain rejects switching noise: frestimate injects at known frequencies and measures response at exactly those frequencies, naturally rejecting ripple at other frequencies. PRBS in time domain mixes everything and relies on ssest to separate dynamics from noise.

4. iL gain verification: Approach 2 includes an explicit post-parfor serial loop that catches and corrects sign errors and >10% magnitude errors in iL DC gain -- a known failure mode when parfor workers log the wrong signal.

5. Forced model structure: tfest(np=2, nz=0) strictly enforces 2nd order with no zeros, matching the physics. ssest with order=2 fits a more general structure that can overfit switching noise.

Key Techniques and Tricks

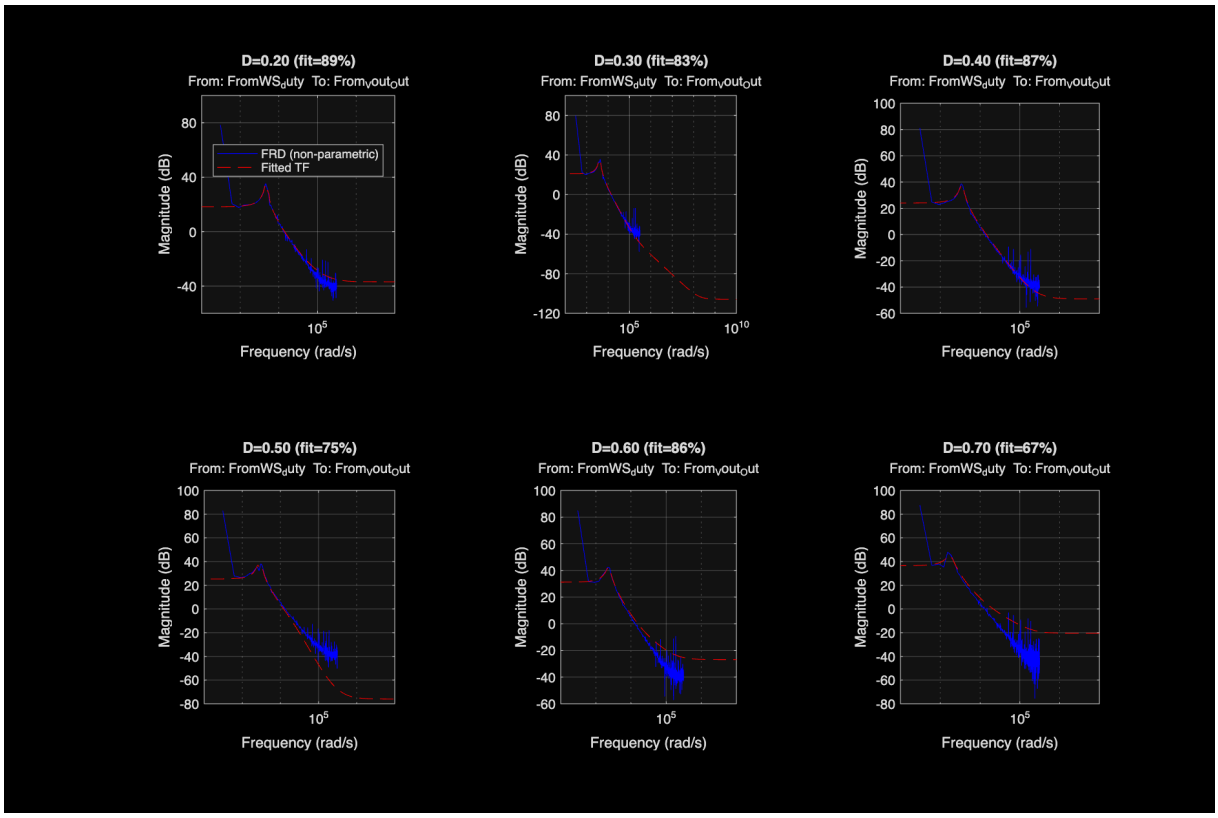
TRICK (Generalizable): Switching Frequency Detection via find_system: Use MATLAB find_system to automatically detect the switching frequency from the Simscape model. Search for PWM Generator, Pulse Generator, and Repeating Sequence blocks, then extract the switching period from block parameters (e.g., get_param(blk, "Period")). This eliminates manual inspection and works for any Simscape model with switching elements.

TRICK (Generalizable): State-Space Order from Energy Storage Elements: The model order for tfest equals the number of independent energy storage elements. For the boost converter: 1 inductor + 1 capacitor = 2nd order (np=2). For any converter, count the inductors and capacitors to determine the correct model order. This prevents overfitting (too high order) or underfitting (too low order) in the system identification step.

TRICK (Generalizable): DC Gain Correction: The identified transfer functions often have incorrect DC gain due to the switching dynamics confusing the frequency response at low frequencies. Correct by running steady-state simulations at each duty cycle and forcing the transfer function DC gain to match the measured steady-state.

TRICK (Generalizable): Conservative tfest Parameters: For stable LPV ROMs, use conservative tfest parameters: np=2 (matching energy storage count), nz=0 (no zeros), and force stability. This prevents overfitting to noise in the frequency response and ensures smooth interpolation across operating points.

TRICK (Generalizable): Geometric Mean Cutoff for Ripple Separation: To separate averaged dynamics from switching ripple, use a filter cutoff at the geometric mean of the LC resonance and switching frequency: f_cutoff = sqrt(f_LC * f_sw). For this converter: sqrt(600 * 200000) = 11 kHz. This cleanly separates the two frequency bands.



LPV Model Construction

The LPV model interpolates between local linear models as the duty cycle (scheduling parameter) varies:

```
x(k+1) = A(D) * x(k) + B(D) * u(k)
y(k)   = C(D) * x(k)
```

where A(D), B(D), C(D) are interpolated from the grid of identified models.

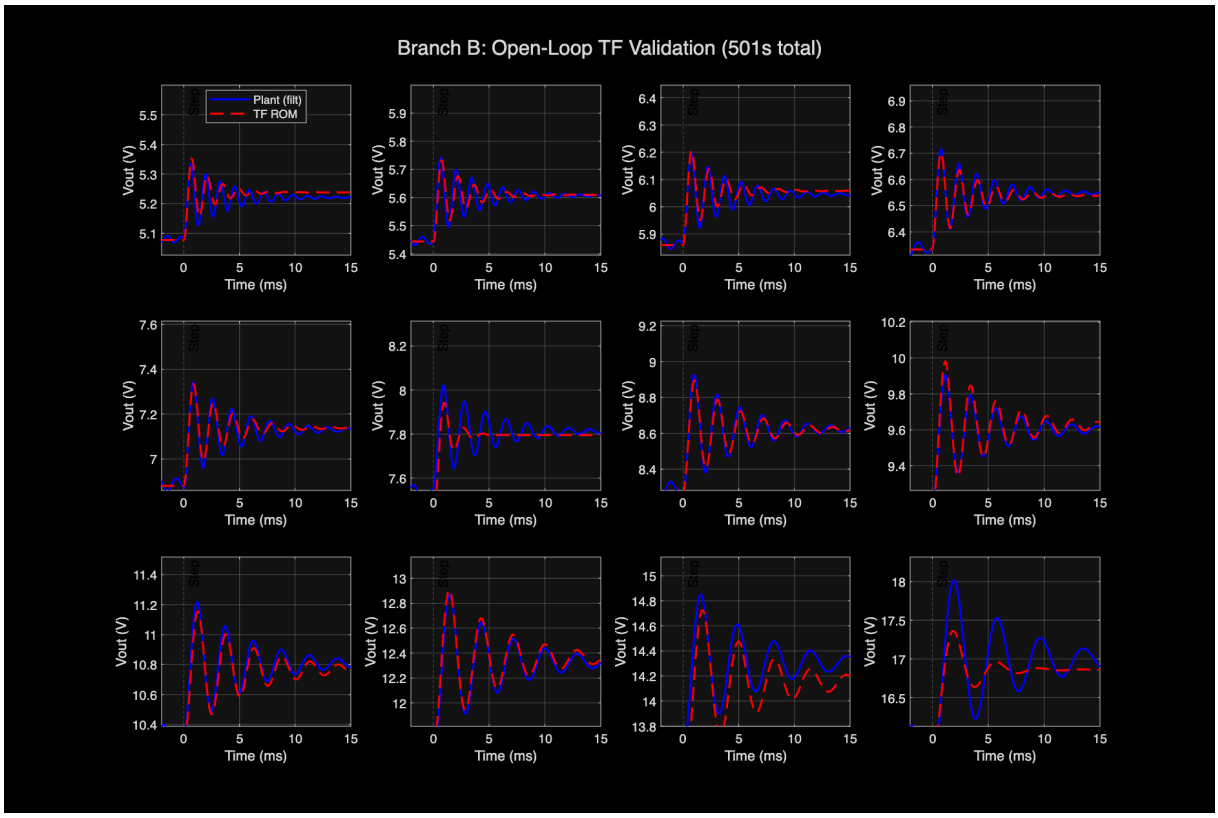
For the inductor current ripple, an analytical triangular reconstruction was added:

```
delta_iL = Vin * D * Tsw / L
```

This formula is specific to the boost converter topology. For general converters, an empirical ripple extraction approach was later developed (see Section 6).

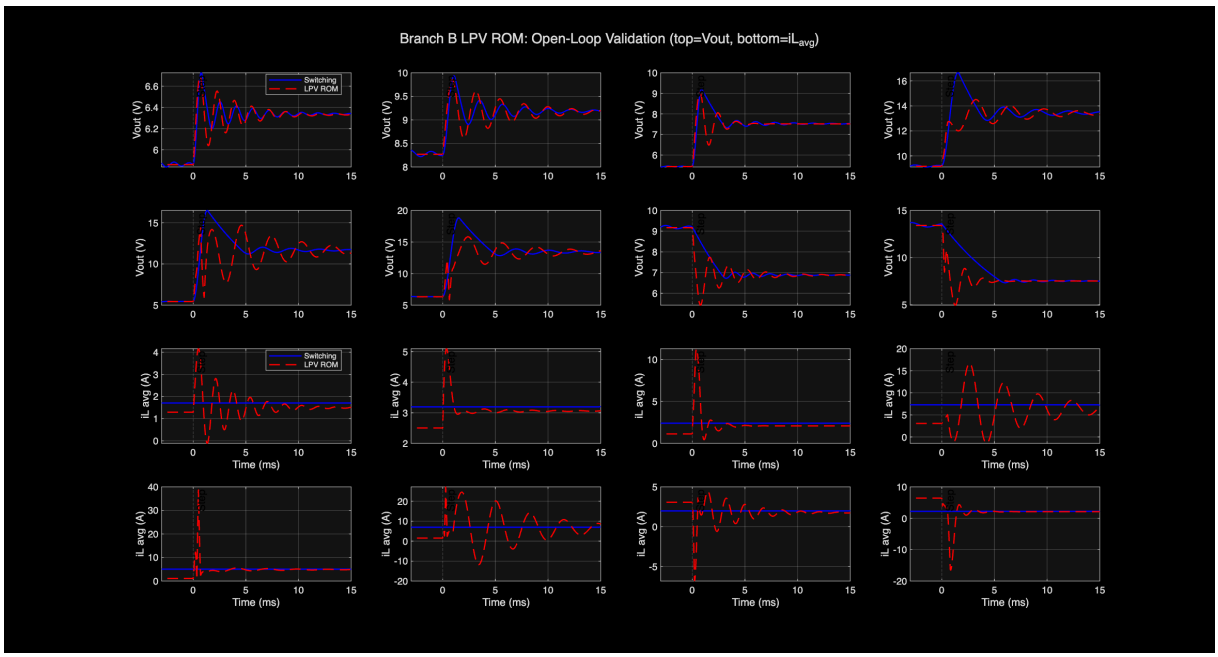
Results

Small-signal validation (small duty cycle steps of +0.02 around each operating point):



The identified linear models match Simscape very well for small perturbations around each operating point. Vout step responses are captured accurately across the full duty cycle range (D = 0.15 to 0.70), confirming that the frestimate+tfest pipeline produces accurate local linear models.

Large-transient validation (duty cycle steps of 0.10 to 0.45, across operating points):



For large duty cycle steps the LPV model breaks down. The top rows (Vout) show significant deviations during transients, and the bottom rows (iL_avg) diverge entirely for large excursions. This is the fundamental limitation of the linear interpolation approach: each local model is accurate near its operating point, but interpolating between distant operating points cannot capture the nonlinear coupling between states during large transients.

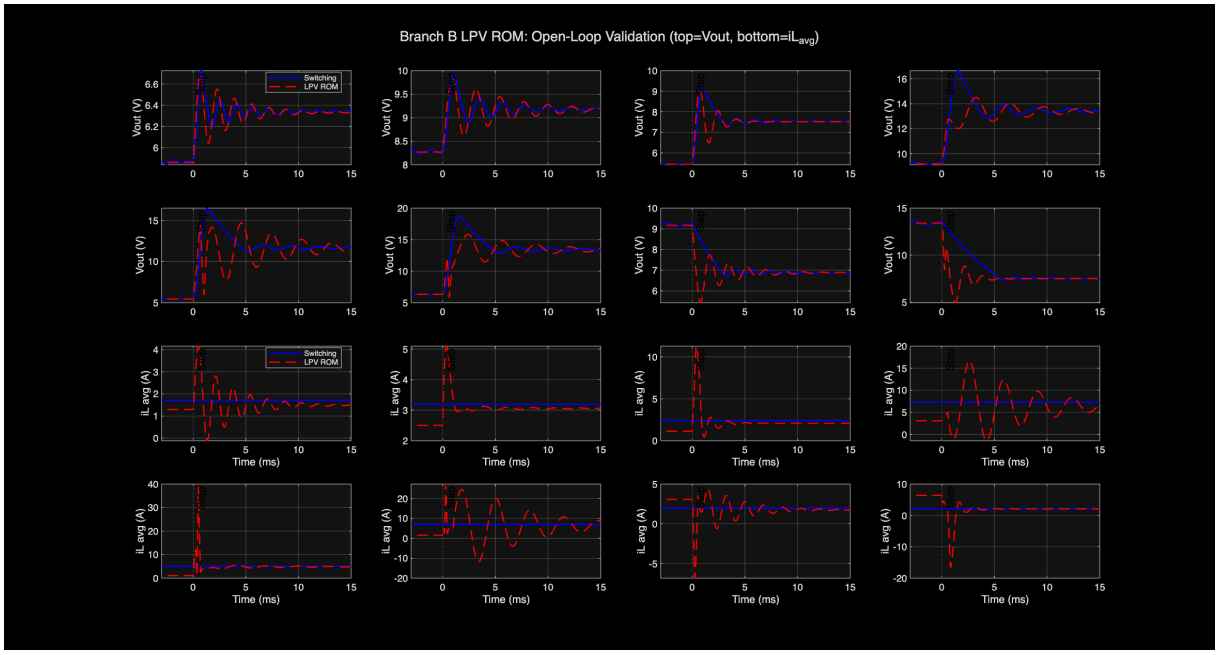
For example, stepping from D=0.20 to D=0.60 takes the converter through a large voltage and current swing that no individual linear model was identified to handle. The interpolated A(D) matrix changes smoothly but the true plant

dynamics change nonlinearly, creating mismatch in both transient shape and settling time.

Limitations

The LPV approach works well for small-signal behavior around each operating point but fails for large-signal transients (e.g., stepping from D=0.2 to D=0.7). The linear models cannot capture the nonlinear coupling between states during large excursions.

This motivated Branch C: a nonlinear Neural ODE that can handle arbitrary operating ranges by learning the full nonlinear dynamics from trajectory data.



Replication Guide

All scripts and data for Branch B are in branch_b_lpv/. To replicate on a new machine:

Step 0 - Setup
  git clone https://github.com/aturevsk/ROM_boost_converter
  Open MATLAB, cd to the repo root, run startup.m

Step 1 - Collect identification data (OPTIONAL - data already provided)
  Script: branch_b_lpv/collect_openloop_id_data.m
  Requires: simulink_models/boost_converter_test_harness.slx
  Method: Applies PRBS perturbations (±5% duty) at 8 operating points
      (D = 0.15, 0.25, 0.35, 0.45, 0.55, 0.65, 0.75, 0.85)
  Runtime: ~30 min (Simscape simulation at each point)
  Output: branch_b_lpv/data/boost_openloop_id_data.mat
  NOTE: Pre-collected data is already provided. Skip this step.

Step 2 - Identify transfer functions with FREST + tfest (OPTIONAL - data provided)
  Script: branch_b_lpv/frest_openloop_id.m
  Method: Parallel frestimate at 12 duty cycle points (D = 0.15 to 0.70),
      fits 2nd-order transfer functions using tfest,
      corrects DC gain via steady-state simulations.
  Requires: Simulink Control Design Toolbox, System Identification Toolbox

```
  Runtime: ~60 min (parallelized over 4 workers)
  Output: branch_b_lpv/models/boost_frest_tf_data.mat
  NOTE: Pre-identified models are already provided. Skip this step.

Step 3 - Build LPV ROM Simulink model
  Script: branch_b_lpv/build_branch_b_lpv_rom.m
  Input:  branch_b_lpv/models/boost_frest_tf_data.mat
  Output: branch_b_lpv/models/boost_openloop_branch_b.slx
          branch_b_lpv/models/boost_openloop_branch_b_data.mat
  Runtime: <1 min

Step 4 - Validate LPV ROM against Simscape
  Script: branch_b_lpv/validate_branch_b_openloop.m
  Input:  branch_b_lpv/models/boost_openloop_branch_b_data.mat
  Runs 8 duty step profiles through both Simscape and LPV ROM,
  prints RMSE table and saves validation plots.
  Runtime: ~10 min

Toolbox Requirements:
  - Simscape Electrical (for boost_converter_test_harness.slx)
  - System Identification Toolbox (ssest, tfest, iddata)
  - Simulink Control Design Toolbox (frestimate) - Steps 1-2 only
```

4. Branch C: PyTorch Neural ODE

4.1 Architecture

The Neural ODE learns a continuous-time state-space model:

```
dx/dt = dxdt_scale * MLP(x_n, u_n)
```

where:
- x = [Vout, iL] are the two physical states
- u = duty cycle (scalar input)
- MLP: [64, 64] hidden units with tanh activation (4,546 parameters)
- x_n = (x - x_mean) / x_std are normalized states
- u_n = (u - u_mean) / u_std is normalized input
- dxdt_scale = [671, 5886] scales MLP output to physical dx/dt

Deployment uses Forward Euler integration at Ts = 5 us:

```
x(k+1) = x(k) + Ts * f(x(k), u(k))
```

KEY INSIGHT: Separate input and output scaling is critical. The MLP with tanh activation produces outputs in ~[-1, 1]. The dxdt_scale buffer (non-trainable) maps this to physical derivatives [~671 V/s, ~5886 A/s]. Without this, the network cannot represent the magnitude of the derivatives, which was the #1 reason MATLAB NSS underperformed. This pattern (normalize inputs, scale outputs) is generalizable to any Neural ODE.

4.2 Normalization Details

Normalization statistics computed from training data (15 profiles):

```
duty:  mean=0.399, std=0.149
Vout:  mean=8.19 V, std=2.69 V
iL:    mean=2.96 A, std=3.35 A
dVout/dt std: 671 V/s
diL/dt std:   5,886 A/s
```

The dxdt_scale is registered as a PyTorch buffer (not a parameter), so it is never modified during training. This ensures the MLP always operates in a well-conditioned O(1) range while the fixed buffer handles the magnitude gap.

KEY INSIGHT: Data normalization was the single most important factor for good training results. An ablation study showed that with proper normalization, even plain MSE loss achieves similar accuracy to the full custom loss (Jacobian + ripple filtering). Without normalization, no amount of training epochs or custom loss helps. This applies to any Neural ODE, not just boost converters.

4.3 Curriculum Training

Training uses a 3-phase curriculum that progressively increases difficulty:

Phase	Epochs	Window	LR	Jacobian	Ripple	Purpose
1 HalfOsc	300	5ms	1e-3	0.1	0	Learn basic dynamics
2 FullOsc	500	20ms	5e-4	0.01	0.1	Learn ripple + longer horizon
3 Finetune	200	20ms	1e-4	0	0.1	Polish

TRICK (Generalizable): Curriculum window sizing: Phase 1 uses 5ms windows (~3 oscillation periods of the dominant 600 Hz mode). This is the "easy" problem: learn one transient event. Short integration windows prevent gradient vanishing/explosion. Phase 2 increases to 20ms (12 oscillation periods), forcing the model to learn self-correcting dynamics. For any system, start with 2-3x the dominant period, then extend to 10-15x.

Integration during training uses fixed-step RK4 with dt = 50 us (step_skip=10 * Ts=5us). This is critical: every 50 us step is computed deterministically, ensuring the model sees all dynamics including LC ringing. The input is held constant within each step (ZOH).

Each epoch processes all 15 training profiles, sampling 3 random windows per profile. Windows start at random positions, preventing overfitting to specific segments.

4.4 Custom Loss Functions

The training loss combines up to three components:

L = L_trajectory + lambda_J * L_jacobian + lambda_ripple * L_ripple

Trajectory loss: MSE between predicted and true states in normalized space.

Jacobian loss: Matches the model linearization (dF/dx) at steady-state operating points to A-matrices identified from Branch B. This ensures the model has physically correct small-signal dynamics (resonant frequency, damping) at each duty cycle.

Ripple loss: Separates prediction error into low-pass (averaged dynamics) and high-pass (switching ripple) components using Butterworth filters at the geometric mean cutoff. Allows independent weighting of ripple accuracy.

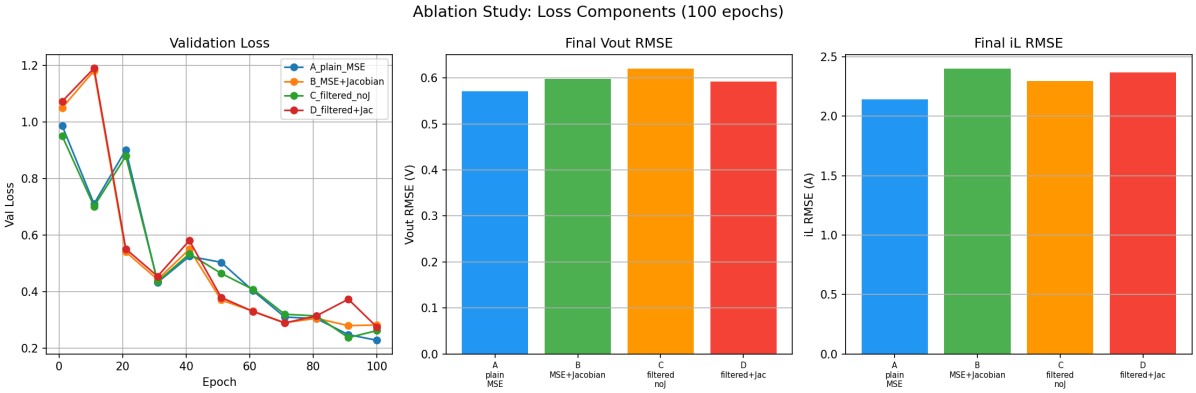
Generalizability: The Jacobian loss requires pre-identified linear models (from Branch B). The ripple loss requires a known frequency separation. Both are generalizable to any system where local linearizations or frequency bands can be identified.

4.5 Ablation Study

To determine which loss components matter, we ran a controlled ablation study with 4 variants sharing the same Phase 1 weights, then diverging in Phase 2:

Variant	Jacobian	Ripple	Val Loss	Vout RMSE	iL RMSE
A: Plain MSE	No	No	0.189	0.483 V	1.968 A
B: MSE+Jacobian	Yes	No	0.180	0.475 V	1.914 A
C: Filtered	No	Yes	0.193	0.552 V	1.962 A
D: Full	Yes	Yes	0.195	0.500 V	1.993 A

KEY INSIGHT: All 4 variants perform within 7% of each other. The Jacobian term provides a marginal 5% improvement; the ripple filtering does not help. This confirms that proper normalization, not custom loss functions, is the key to good Neural ODE training. Conclusion: invest effort in normalization first, custom losses second.

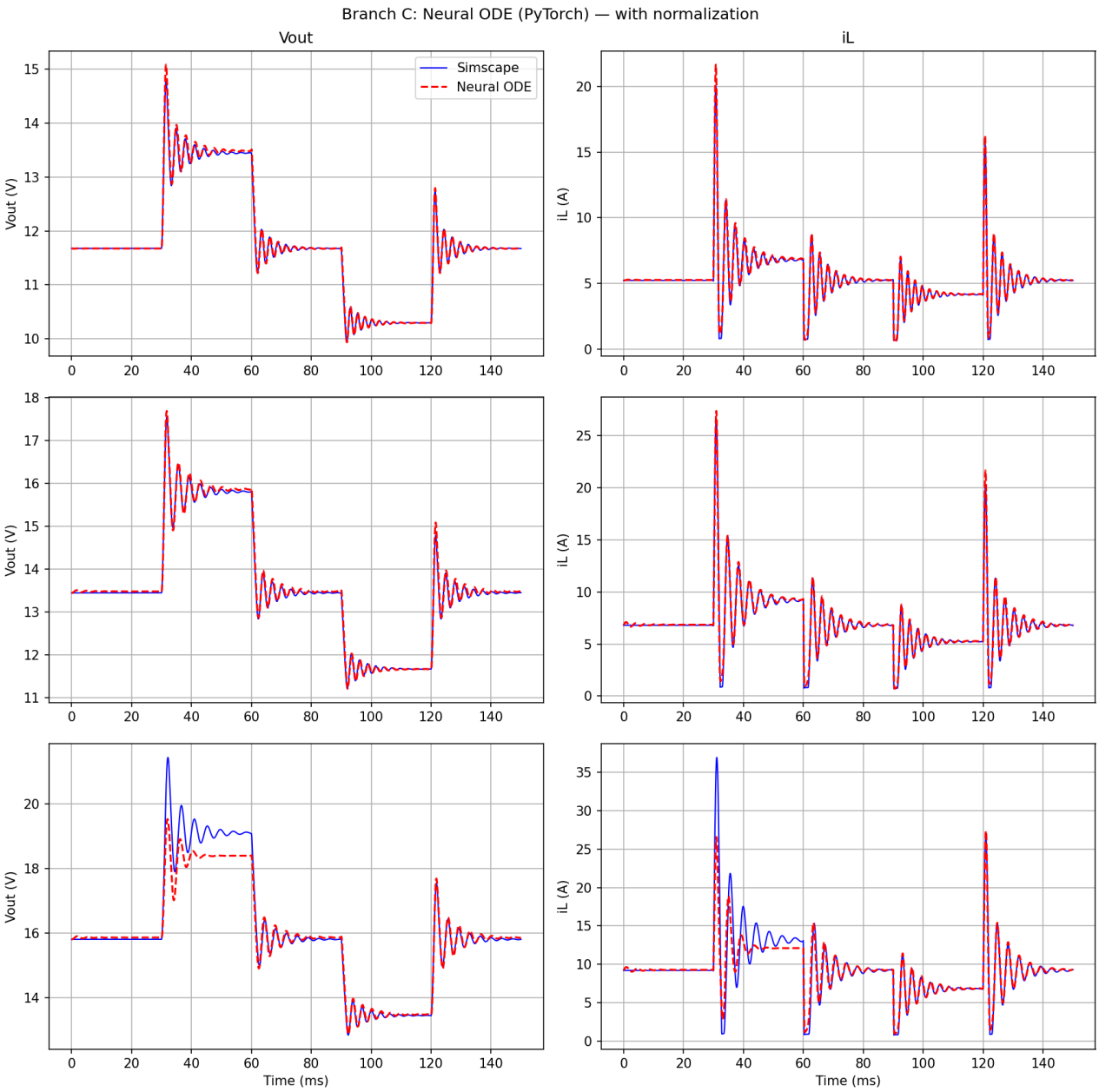


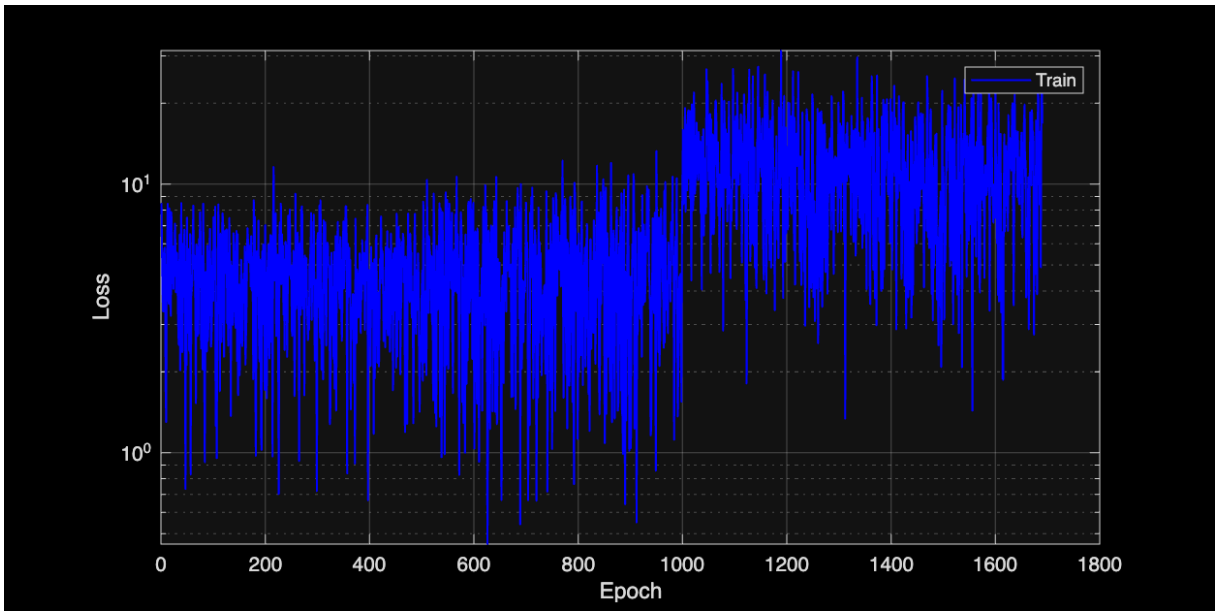
4.6 Extended Training & Final Results

After the 3-phase curriculum (1000 epochs, ~25 min), extended fine-tuning ran for 7.7 hours with ReduceLROnPlateau (starting at LR=2e-4, decaying to 2.5e-5). Checkpoints saved every 15 minutes.

The val loss improved from 0.0878 to 0.0401 during extended training, a 54% reduction.

Profile	Vout RMSE	iL RMSE
Val 0 (easy)	0.047 V	0.214 A
Val 1 (medium)	0.064 V	0.289 A
Val 2 (hard)	0.386 V	1.524 A


Branch C: Neural ODE (PyTorch) — with normalization
Vout
iL
Simscape
Neural ODE
Vout (V)
iL (A)
Time (ms)


Train
Loss
Epoch

5. MATLAB Neural State Space

5.1 Motivation

MATLAB Neural State-Space (idNeuralStateSpace + nlssest) provides a native MATLAB alternative to the PyTorch Neural ODE. Same MLP architecture [64,64] tanh with 4,546 parameters, same training data, same normalization. The question: can MATLAB System Identification Toolbox match PyTorch accuracy?

5.2 Five Issues vs PyTorch

Root cause analysis identified five key differences:

Issue 1: Adaptive ODE Solver (HIGH IMPACT)

PyTorch uses fixed-step RK4 (dt=50us, exactly 400 steps per 20ms window). MATLAB uses adaptive dlode45 with AbsTol=RelTol=0.01 (very loose!). The adaptive solver can skip steps during quasi-steady-state and miss fast transients. PyTorch resolves every 50us regardless.

Issue 2: Output Scaling (MEDIUM IMPACT)

PyTorch has a non-trainable dxdt_scale buffer that keeps the MLP output in O(1) range. MATLAB NSS has no equivalent mechanism. We attempted to scale the last layer weights at initialization, but these weights are free to drift during training, making optimization harder.

Issue 3: Training/Validation Mismatch (MEDIUM IMPACT)

PyTorch validates using Forward Euler at 5us (same as deployment). MATLAB validates using dlode45, which is NOT what the Simulink Neural State Space block uses at runtime. The "best" model is optimized for the wrong integration scheme.

Issue 4: No Curriculum Control (LOW IMPACT)

nlssest is a black box: no per-window gradient updates, no Jacobian loss, no gradient clipping, no curriculum phases beyond WindowSize/LR changes.

Issue 5: One-Step-Ahead for Discrete Time

Discrete-time NSS uses one-step-ahead prediction (teacher forcing), not free-running simulation. The model never learns to correct its own drift during training. This was discovered from the nlssest documentation: the loss function is V(theta) = 1/N * sum(epsilon^2), where epsilon is the per-step prediction error.

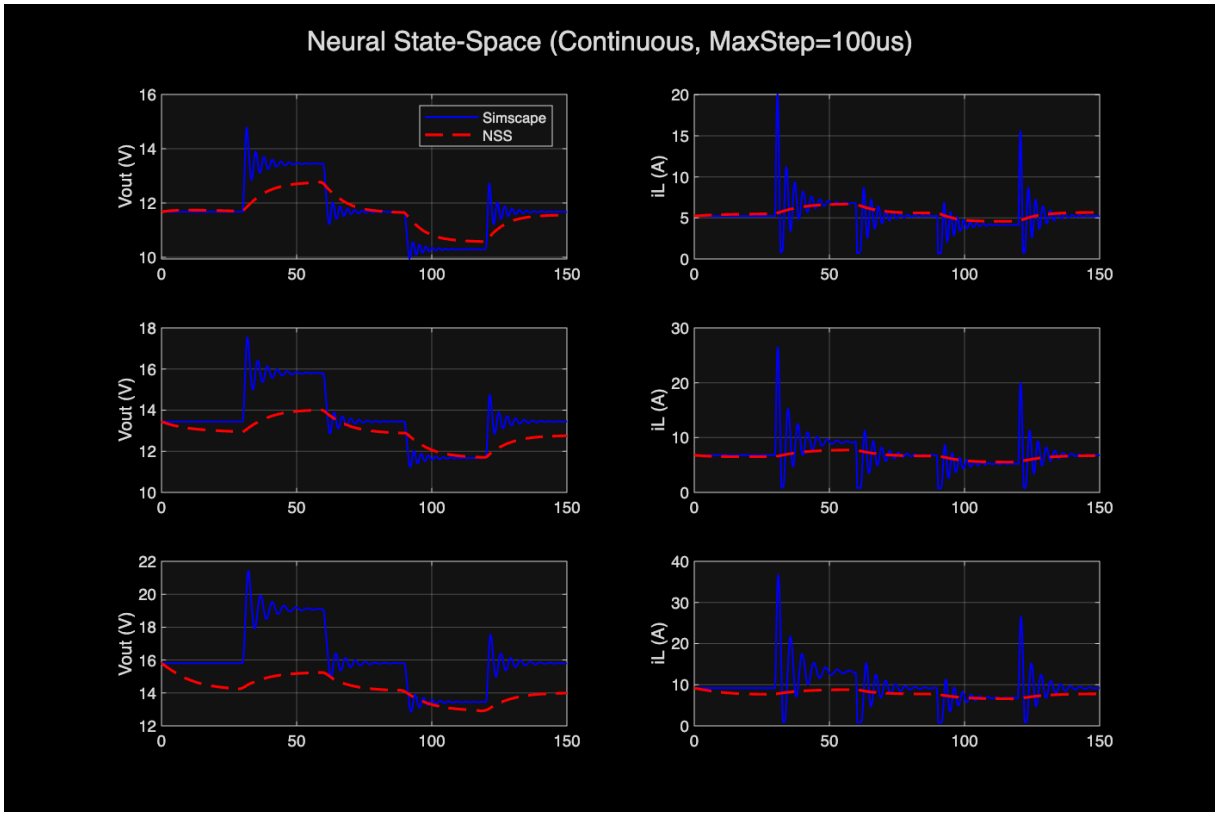
KEY INSIGHT: The combination of loose solver tolerances + no output scaling + teacher forcing means MATLAB NSS was fundamentally training a different problem than PyTorch. PyTorch trains a free-running ODE integrator with proper scaling; MATLAB trains a one-step predictor with an imprecise ODE solver. These findings apply to any MATLAB NSS vs PyTorch Neural ODE comparison.

5.3 What Was Tried

Approach	Val Loss	Vout RMSE	Result
CT + default (baseline)	0.168	0.60 V	No dynamics
CT + output-scaled init	0.137	0.47 V	Better, still misses ringing
CT + MaxStepSize=50us	0.137	0.47 V	No additional improvement
CT + L-BFGS optimizer	0.217	0.47 V	Made it worse (diverged)
DT + skip connection	0.339	0.45 V	Teacher forcing limits learning
DT + 3-phase + L2 reg	0.339	0.45 V	Same ceiling
PyTorch (reference)	0.040	0.05 V	3.4x better

Best MATLAB NSS result: val loss 0.137 after 1750 epochs of Adam training + L-BFGS fine-tuning (total ~2 hours). This is 3.4x worse than PyTorch (0.040).

The continuous-time approach (with output-scaled initialization) performed best. The discrete-time fix attempt, despite addressing the ODE solver issue, performed worse because nlssest uses teacher forcing for discrete-time models, preventing the network from learning to correct its own prediction drift.



KEY INSIGHT: For MATLAB Neural State Space to match PyTorch, a custom training loop with dlfeval/dlgradient would be needed, implementing free-running trajectory loss with fixed-step integration. This would essentially replicate the PyTorch training loop in MATLAB, making the NSS framework unnecessary. The existing MATLAB train_neural_ode.m script does this but is ~8x slower than PyTorch due to MATLAB interpreter overhead.

5.4 Expert Training: NumWindowFraction=eps

An expert suggestion was to set NumWindowFraction=eps in nssTrainingOptions, which forces nlssest to update network parameters after EVERY data window (stochastic gradient descent) rather than accumulating gradients over the full batch. This mimics PyTorch Adam's per-step updates and is the closest MATLAB NSS can get to PyTorch's training

regime without writing a custom training loop.

Script: matlab_neural_ss/train_nss_expert.m

Training Configuration

Parameter	Value	Rationale
NumWindowFraction	eps (stochastic)	Update after every window (like PyTorch Adam)
Phase 1	300 epochs, 5ms windows	Short windows for initial convergence
Phase 2	500 epochs, 20ms windows	Longer windows for trajectory accuracy
Phase 3	200 epochs, 20ms, LR=1e-4	Fine-tuning with reduced learning rate
Val check frequency	Every 25 epochs	Monitor progress during training
Early stopping	100 epochs no improvement	Prevent wasted compute on plateaus
Checkpoint frequency	Every 15 min + new best	Preserve best model during long runs

Results

Approach	Val Loss	vs Baseline	vs PyTorch
CT + output-scaled init (baseline)	0.137	---	3.4x worse
CT + NumWindowFraction=eps (expert)	0.130	5% improvement	3.25x worse
PyTorch Neural ODE (reference)	0.040	3.25x better	---

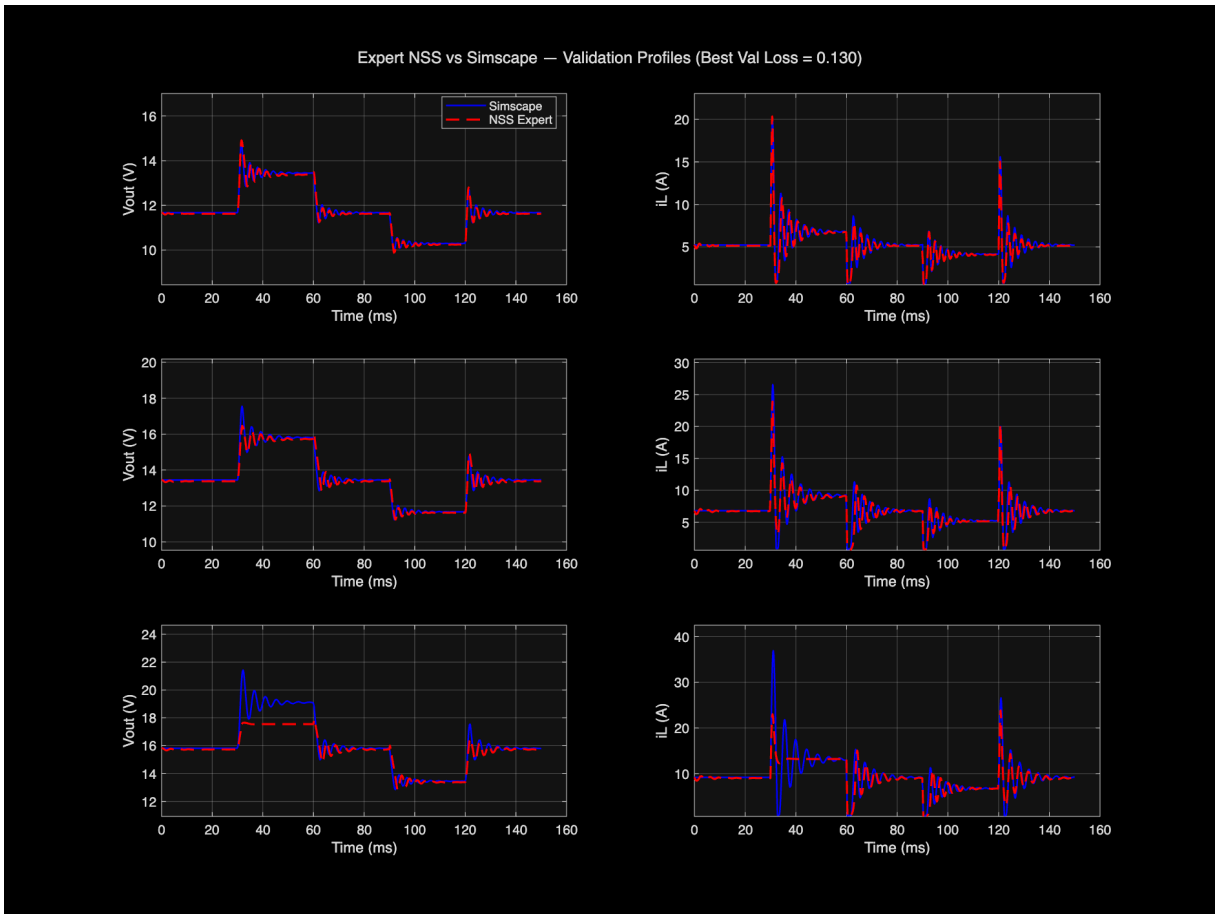
The expert training achieved val loss 0.130, a 5% improvement over the baseline 0.137. While statistically meaningful, the gap with PyTorch (0.040) remains large (3.25x).

The remaining gap is structural, not a training hyperparameter issue:

1. Solver cost: MATLAB uses adaptive dlode45 (~4.2 min/epoch); PyTorch uses fixed-step RK4 (~22 sec/epoch, 11x faster per epoch).

2. Custom loss: PyTorch applies a dual loss (L_avg + lambda_ripple * L_ripple) with separate low-pass and high-pass MSE components. nlssest cannot accept a custom loss function - the ripple-aware loss is a structural advantage of the PyTorch approach.

3. Free-running training: PyTorch trains on full 20ms trajectories integrated with fixed-step RK4. nlssest uses dlode45 with adaptive steps, which does not match the deployment solver and introduces solver-induced gradient noise.



KEY INSIGHT: NumWindowFraction=eps is the best available MATLAB NSS training option and yields a small but real improvement (0.137 -> 0.130). The remaining 3.25x gap vs PyTorch is structural: custom loss functions and solver choice are not configurable in nlssest. The expert NSS model (val=0.130) is used in the comparison simulation.

5.5 DLT Custom Training Loop (V1/V2/V3)

To close the gap with PyTorch, we bypassed nlssest entirely and built a custom training loop using Deep Learning Toolbox (dlfeval, dlgradient, adamupdate). This allowed replicating the PyTorch training pipeline component by component: same MLP [3->64->tanh->64->tanh->2], same RK4 integrator (dt=50us), same custom loss (LP/HP filtered MSE + Jacobian matching), same 3-phase curriculum, same Adam optimizer with gradient clipping. Three variants were tested:

V1: Manual RK4 + Constant LR

Exact replication of PyTorch training: manual RK4 integration at dt=50us with direct backpropagation through the unrolled loop via dlgradient. dlaccelerate used for JIT compilation. Only difference from PyTorch: no LR schedule (constant per phase).

Script: matlab_neural_ss/train_dlt_neural_ode.m

V2: dlode45 Adjoint + Cosine LR

Hypothesis: gradient degradation through 400 unrolled RK4 steps limits 20ms window accuracy. Fix: replace manual RK4 with dlode45(GradientMode="adjoint") which computes gradients via a reverse-time ODE (no gradient chain). Also added cosine annealing LR matching PyTorch CosineAnnealingLR.

Result: windowed val loss was excellent (0.0008) but full-profile accuracy was WORST of all DLT variants (1.41V Vout RMSE). The adaptive ode45 solver created a train-deploy mismatch -- the model learned dynamics specific to adaptive stepping that do not transfer to fixed-step RK4 deployment.

Script: matlab_neural_ss/train_dlt_neural_ode_v2.m

V3: Manual RK4 + Cosine LR

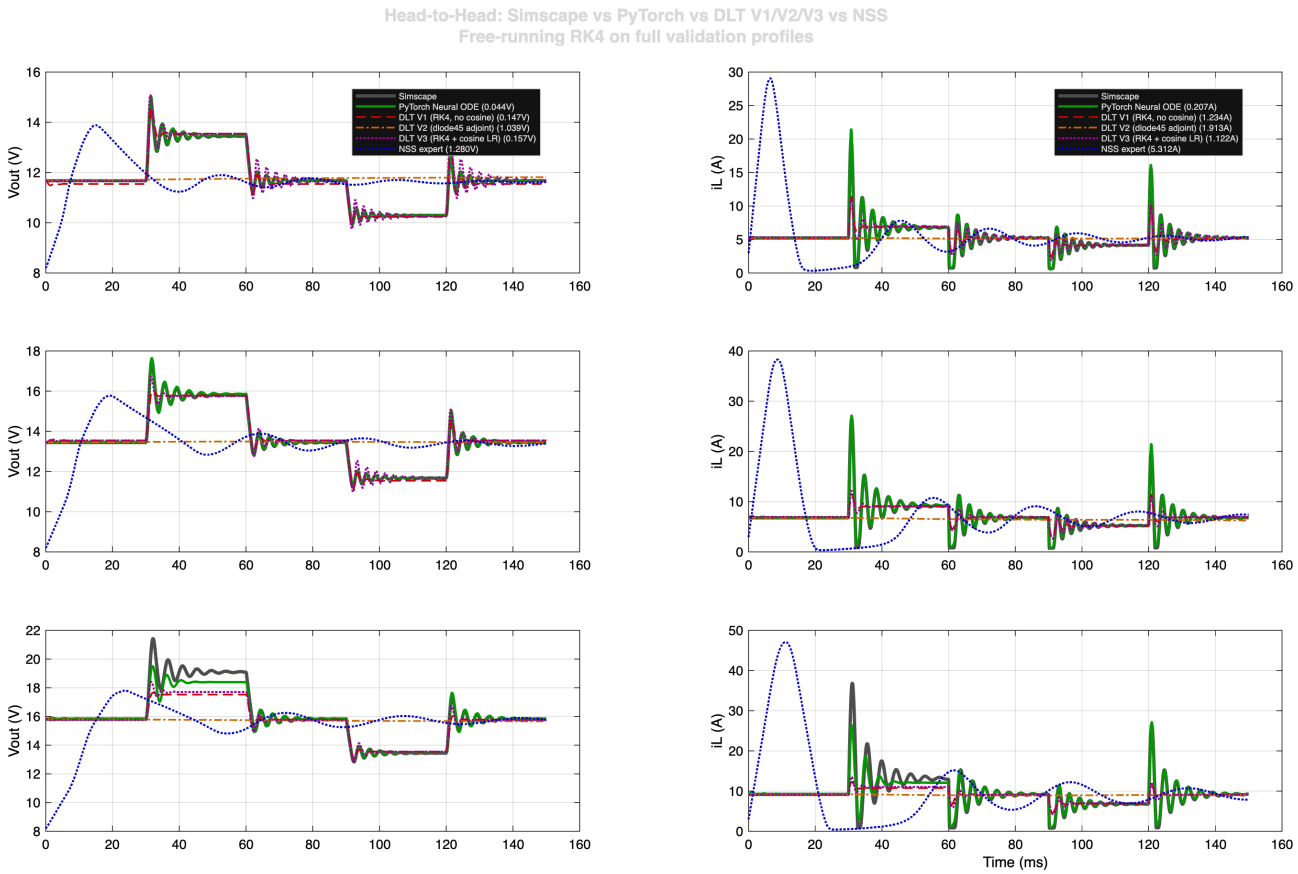
V1 approach (proven RK4) plus cosine LR schedule (the one feature V1 was missing vs PyTorch). This isolated the LR schedule as a variable.

Result: marginal improvement over V1 (0.342V vs 0.387V Vout RMSE). Cosine LR provides ~10%% improvement but is not the primary factor explaining the gap with PyTorch.

Script: matlab_neural_ss/train_dlt_neural_ode_v3.m

Head-to-Head Results (Full-Profile Free-Running)

Model	Vout RMSE	iL RMSE	Training Time
PyTorch Neural ODE	0.163 V	0.673 A	7.7 h
DLT V3 (RK4 + cosine LR)	0.342 V	2.099 A	3.4 h
DLT V1 (RK4, constant LR)	0.387 V	2.233 A	4.5 h
DLT V2 (dlode45 adjoint)	1.410 V	2.928 A	0.8 h
NSS expert (nlssest)	1.833 V	8.212 A	8 h



KEY INSIGHT: The DLT custom training loop is 4-5x better than nlssest and trains faster than PyTorch (3.4h vs 7.7h, with Phase 1 running 15-37x faster via dlaccelerate). However, a 2x accuracy gap vs PyTorch remains even after matching all training components exactly. The adjoint method (V2) backfired due to solver mismatch. Cosine LR (V3) helped only marginally. The remaining gap is likely in autograd numerical differences between PyTorch and MATLAB dlgradient for long computation chains.

What Was Matched vs PyTorch

Component	Matched?
MLP architecture [3->64->tanh->64->tanh->2]	Yes
RK4 integrator (dt=50us, step_skip=10)	Yes
Custom loss (LP/HP filtered MSE)	Yes
Jacobian matching loss	Yes
3-phase curriculum (5ms/20ms windows)	Yes
Adam optimizer + gradient clipping	Yes
Cosine annealing LR (V3)	Yes
Normalization (same JSON stats)	Yes
Train/val split (15/3 profiles)	Yes

Lessons Learned

1. Custom training loops in MATLAB DLT are viable and can be faster than PyTorch for small models, thanks to dlaccelerate JIT compilation.

2. The solver used during training must match deployment. dlode45 adjoint gives excellent windowed accuracy but the

model does not transfer to fixed-step RK4.

3. nlssest is fundamentally limited by its lack of custom loss functions and free-running integration during training. A custom loop with the same MLP is 4-5x better.

4. A ~2x accuracy gap between MATLAB DLT and PyTorch persists even with identical training configurations. This warrants further investigation into autograd numerical differences for long backpropagation chains.

6. PyTorch to Simulink Deployment

Pipeline

Converting the trained PyTorch Neural ODE to a Simulink model follows this pipeline:

1. torch.jit.trace() -> TorchScript (.pt file)
2. importNetworkFromPyTorch() -> MATLAB dlnetwork
3. exportNetworkToSimulink() -> Simulink layer blocks
4. Build Simulink model with integration loop:

```
[Normalize] -> [MLP] -> [Scale by dxdt_scale] -> [Euler] -> x_state
              ^                                  |
              +--- state feedback (Unit Delay) -------------+
```

Two Simulink model variants were created:

Model A (Predict block): Uses the dlnetwork directly via the Deep Learning Predict block. Supports SimulateUsing = Code generation for acceleration.

Model B (Layer blocks): Uses exportNetworkToSimulink to create individual Simulink blocks for each layer (FC, Tanh, FC, Tanh, FC). More transparent, easier to debug.

TRICK (Generalizable): PyTorch Import Fix: importNetworkFromPyTorch imports the TorchScript model but creates an uninitialized dlnetwork with auto-generated layer names (e.g., "aten_layer_1"). The trick: (a) build a fresh dlnetwork with a proper featureInputLayer and named layers (fc1, tanh1, fc2, tanh2, fc3), (b) extract weight matrices from the imported network, (c) copy them into the fresh network. This ensures proper layer naming for exportNetworkToSimulink and makes the Simulink model readable.

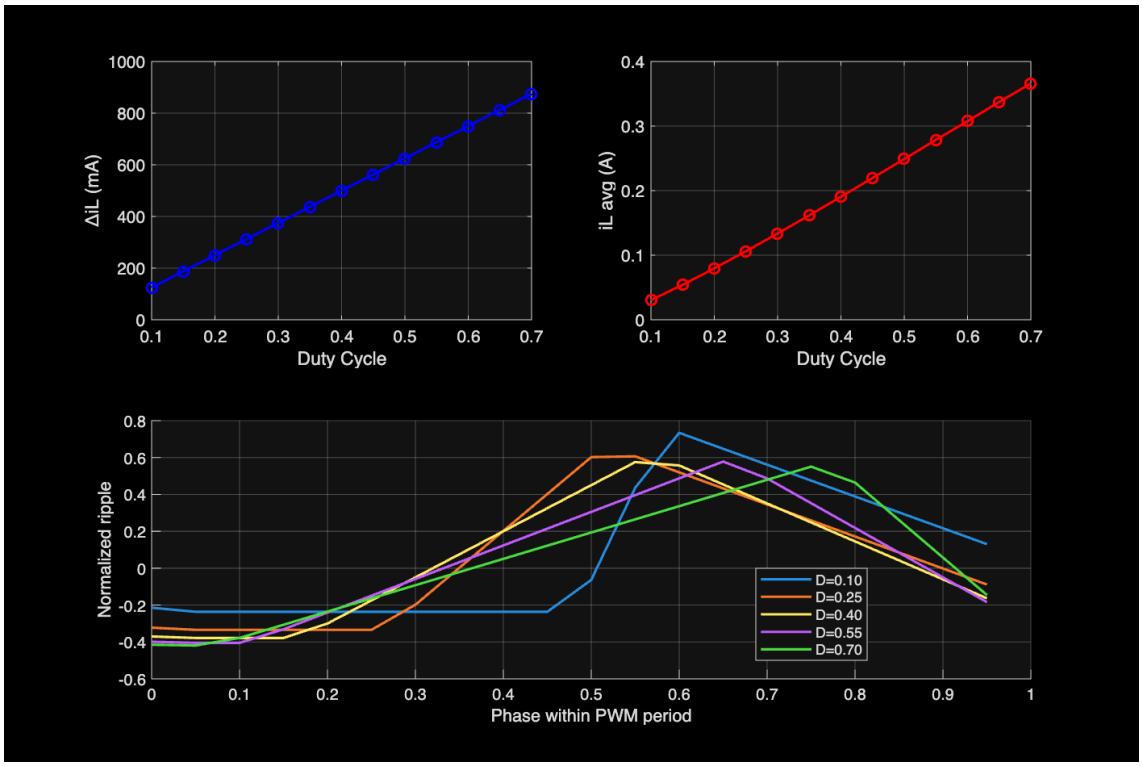
TRICK (Generalizable): Normalization placement: The MLP operates in normalized space, but the Euler integration and state feedback operate in PHYSICAL space. Normalization must be applied at the MLP input, and the dxdt_scale multiplication + Euler step happen after denormalization. This matches exactly how PyTorch computes: dxdt = dxdt_scale * MLP(norm(x), norm(u)). This pattern applies to any Neural ODE deployed in Simulink.

Ripple Reconstruction

The Neural ODE predicts averaged inductor current (iL_avg). To reconstruct switching ripple, an empirical approach was developed:

1. Run Simscape at multiple steady-state duty cycles
2. Extract iL waveform at full resolution (sub-switching period)
3. Measure peak-to-peak ripple amplitude vs duty cycle
4. Build a 1-D lookup table: duty -> delta_iL
5. Reconstruct triangular ripple: iL = iL_avg +/- delta_iL/2

This approach is topology-agnostic: it measures rather than computes the ripple, so it works for any converter topology without knowing the inductor voltage equation.



*Generalizability: The empirical ripple extraction works for any switching converter. The analytical formula (delta_iL = Vin*D*Tsw/L) is boost-specific, but the lookup-table approach requires no topology knowledge.*

7. Comparison & Timing Results

Simulation Speed

All models were benchmarked on the same 1.4-second staircase duty profile:

Model	Sim Time	Speedup	Vout RMSE	Val Loss
Simscape (reference)	258 s	1x	---	---
PyTorch ROM (Layer blocks)	5.5 s	47x	< 0.05 V	0.040
PyTorch ROM (Predict block)	6.8 s	38x	< 0.05 V	0.040
MATLAB NSS ROM (expert)	2.2 s	120x	~0.44 V	0.130

KEY INSIGHT: Layer blocks run 24% faster than the Predict block (5.5s vs 6.8s) because they compile into native Simulink execution, avoiding dlnetwork inference overhead. MATLAB NSS (expert-trained, val=0.130) is fastest at 2.2s (120x) but with ~9x worse val loss than PyTorch. The PyTorch ROM (Layer blocks) achieves the best speed-accuracy tradeoff at 47x speedup with <0.05V Vout RMSE.

Test Profile

All four models (Simscape, PyTorch ROM x2, MATLAB NSS ROM) are tested with an identical duty cycle staircase profile:

- Initial hold at D=0.20 for 0.2s
- Staircase UP: 10% steps from D=0.20 to D=0.70 (0.1s per step)
- Staircase DOWN: 10% steps from D=0.70 to D=0.20 (0.1s per step)
- Big transient UP: D=0.20 -> 0.70, hold 0.1s
- Big transient DOWN: D=0.70 -> 0.20, hold 0.1s
- Total: 1.4 s

The duty range was limited to 0.20-0.70 because the training data only covers D=0.125 to 0.75. Values outside this range are extrapolation and show degraded accuracy.

Key Findings

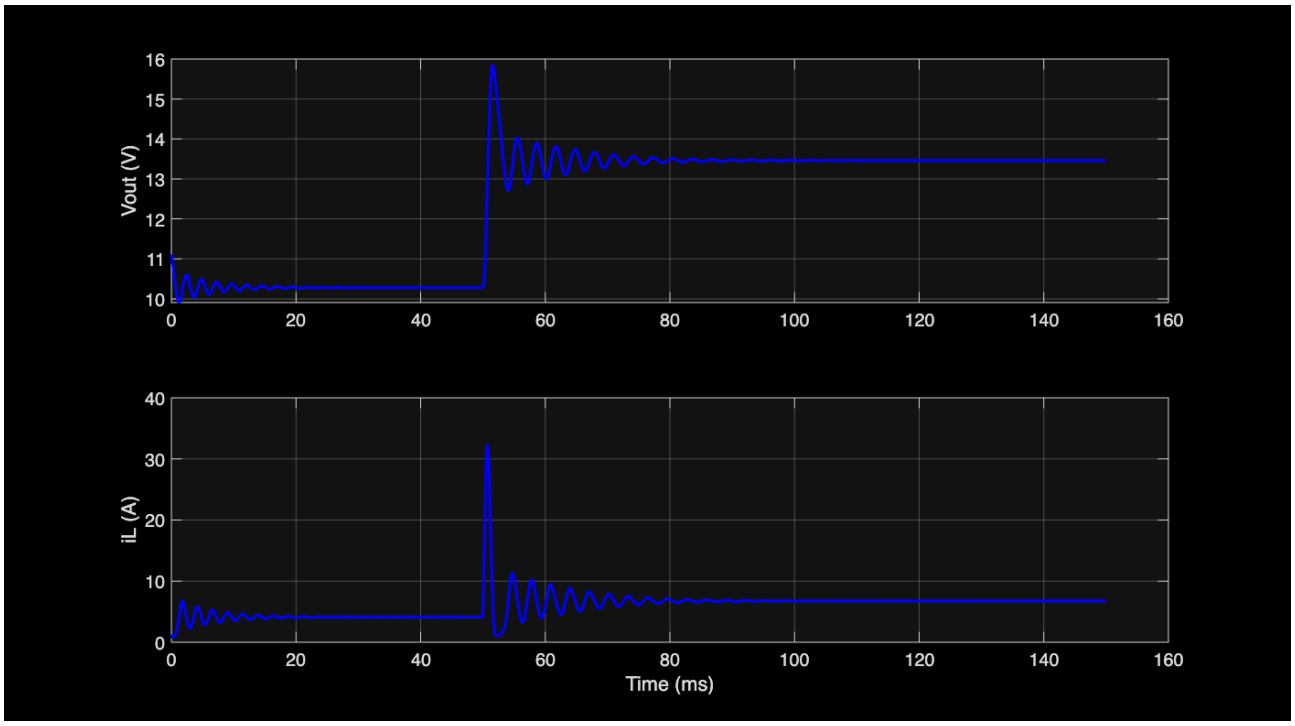
1. PyTorch Neural ODE ROM tracks the Simscape reference closely across all duty steps, including the LC oscillatory transients after each step change.

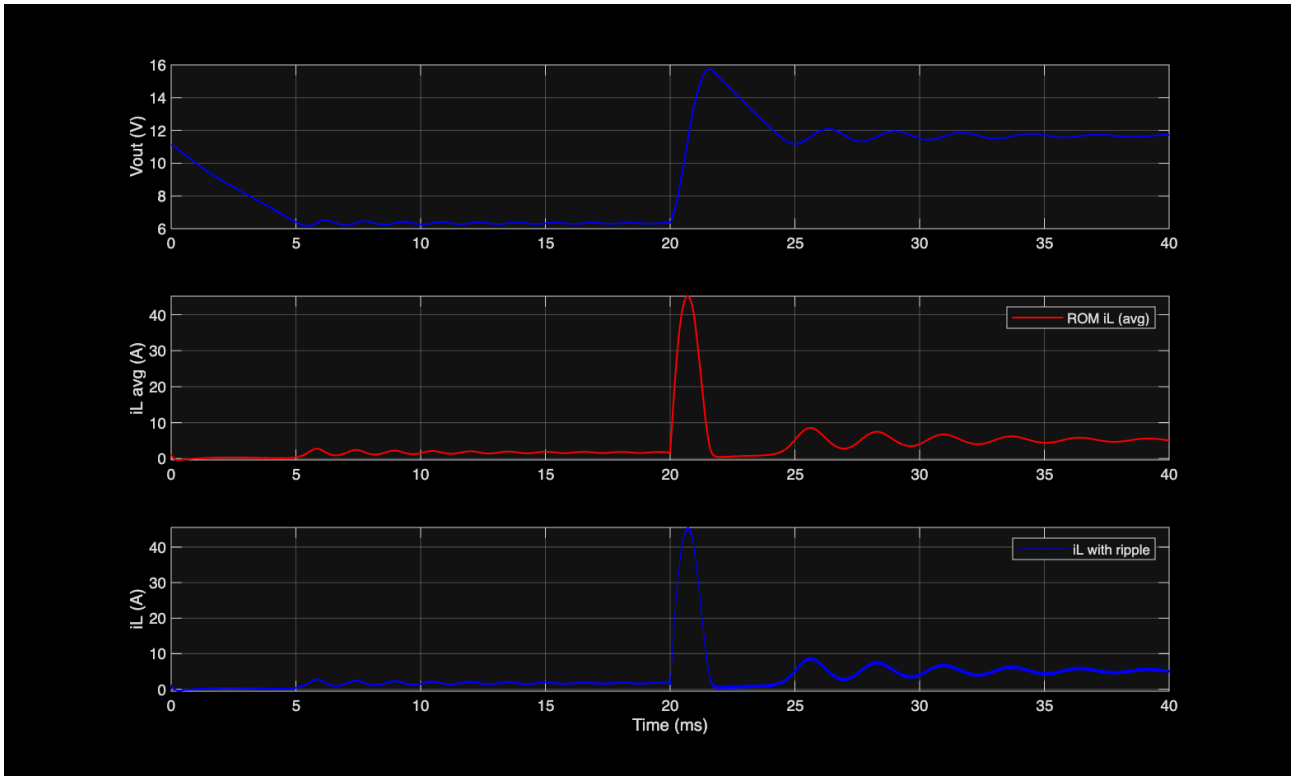
2. MATLAB NSS ROM (expert-trained, val=0.130) captures the correct steady-state levels but has larger transient errors and less accurate ringing behavior.

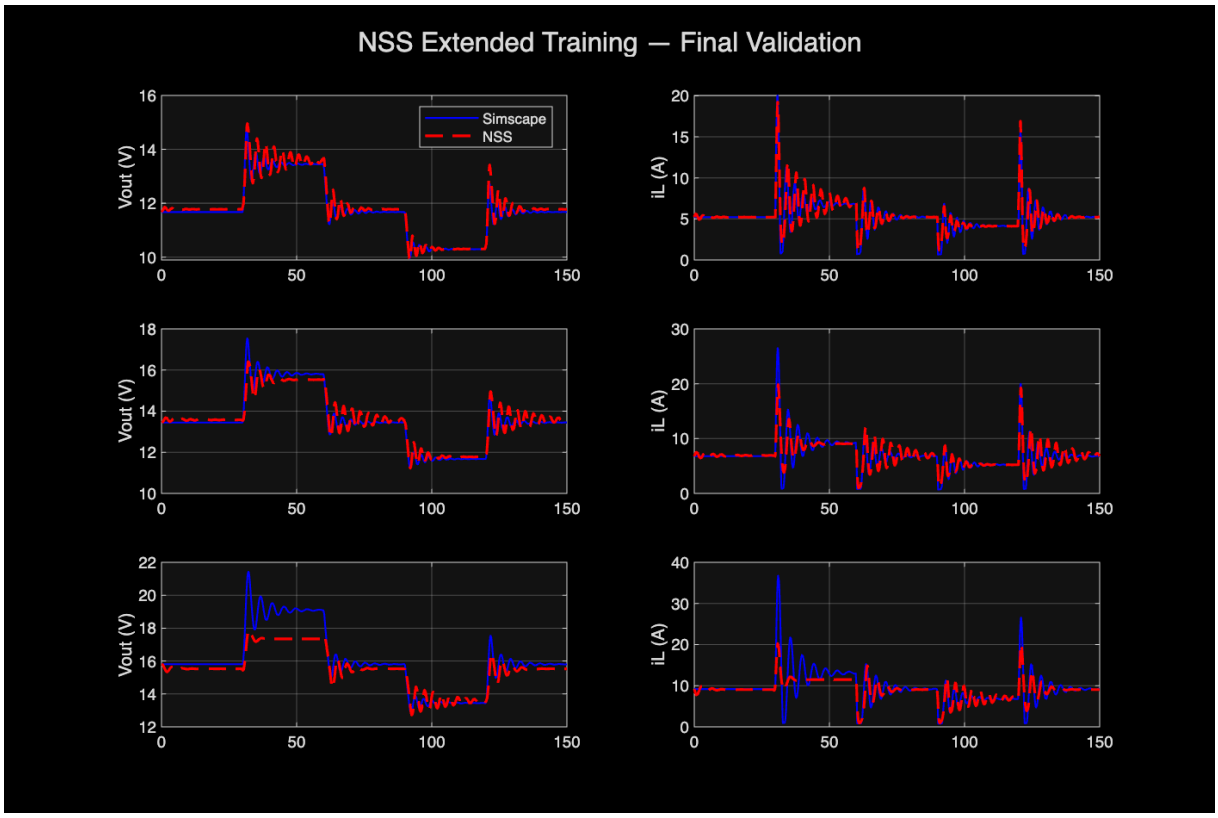
3. All ROMs are dramatically faster than Simscape (47x for PyTorch, 120x for NSS).

4. At the edges of the training range (D=0.20 and D=0.70), both ROMs show slightly more error due to limited training data coverage.

5. To run the comparison yourself: open simulink_models/setup_comparison_profile.m and run it - it configures all four models with the duty profile and opens them for interactive simulation.


Vout (V)
iL (A)
Time (ms)


Vout (V)
iL avg (A)
ROM iL (avg)
iL (A)
iL with ripple
Time (ms)


NSS Extended Training — Final Validation
Simscape
NSS
Vout (V)
iL (A)

8. Repository File Listing

pytorch_neural_ode/

File	Description
train_neural_ode_pytorch.py	Main 3-phase curriculum training (NeuralODE + custom loss)
train_extended.py	8-hour extended fine-tuning with plateau detection
export_neuralode_weights.py	Export checkpoint to JSON + MAT + TorchScript

matlab_neural_ss/

File	Description
train_neural_ss_normalized.m	NSS: CT with output-scaled init (val=0.137)
train_nss_fixed.m	NSS: DT fix attempt with skip connection + 3-phase
train_nss_expert.m	NSS: NumWindowFraction=eps (val=0.130)
train_dlt_neural_ode.m	DLT V1: Custom RK4 + direct backprop (Vout 0.387V)
train_dlt_neural_ode_v2.m	DLT V2: dlode45 adjoint + cosine LR (Vout 1.41V)
train_dlt_neural_ode_v3.m	DLT V3: RK4 + cosine LR (Vout 0.342V, best MATLAB)
compare_all_five.m	Head-to-head comparison of all 5 models

simulink_models/

File	Description
boost_converter_test_harness.slx	Simscape open-loop reference model
boost_openloop_branch_c_layers.slx	PyTorch ROM via Simulink layer blocks
boost_openloop_branch_c_predict.slx	PyTorch ROM via Deep Learning Predict block
boost_openloop_nss.slx	MATLAB NSS ROM via Neural State Space block
setup_comparison_profile.m	Configures all models with same duty profile

model_data/

File	Description
extended_best.pt	Best PyTorch checkpoint (val loss=0.040, 2500 epochs)
neuralode_weights.mat	MLP weights for Simulink deployment
neuralode_dlnetwork.mat	MATLAB dlnetwork object (imported from PyTorch)
neuralode_norm_stats.json	Normalization statistics (means, stds, dxdt_scale)
boost_branch_c_data.mat	ROM data: normalization + ripple lookup tables
boost_nss_normalized.mat	MATLAB NSS baseline model (val loss=0.137)
boost_nss_expert.mat	Expert NSS model, NumWindowFraction=eps (val loss=0.130)
comparison_duty_profile.mat	Staircase duty profile timeseries
ripple_empirical_data.mat	Empirical ripple amplitude vs duty cycle

data/

File	Description
neural_ode/profile_01..18.csv	18 training profiles: [duty, Vout, iL, SimID]
neural_ode/jacobian_targets.json	A-matrices for Jacobian loss at 12 duty points
boost_nss_training_data.mat	Same data in MATLAB iddata format for nlssest